

For each question, provide:

1. The question in words.
2. Your final result.
3. The SQL query used.
4. A short explanation of what the query does and why it answers the question.

Question1:

- What is the total number of applicants for Fall 2026?
- Final Result: 12000
- SQL Query:

```
# Q1
cur.execute("SELECT COUNT(*) FROM applicants WHERE term ILIKE '%fall 2026%';")
q1_count = cur.fetchone()[0] or 0
print(f"Fall 2026 applicant count: {q1_count:,}")
```

- Short explanation:
I use SELECT COUNT() to calculate the total number of rows and filter out the data for the year 2026 using the WHERE function. And I used ILIKE '%fall 2026%' so that it doesn't matter if the text is capital or lowercase, or if there are extra words around it. The second line makes sure that if no data was found, it safely defaults to 0 instead of crashing the code.

Question2:

- What is the percentage of international applicants?
- Final Result: 57.89%
- SQL Query:

```
# Q2
cur.execute("""
SELECT
    100.0 * COUNT(CASE WHEN us_or_international ILIKE '%international%' THEN 1 END) /
    NULLIF(COUNT(CASE WHEN us_or_international IS NOT NULL AND us_or_international != '' THEN 1 END), 0)
FROM applicants;
""")
q2_pct = cur.fetchone()[0] or 0.0
print(f"Percent international: {q2_pct:.2f}%")
```

- Short explanation:
First, I used the 100* to turn it into a percentage and use COUNT(CASE WHEN...) to count how many applicants are marked as international, but do not delete the other data. And I used the result as numerator and use another COUNT(CASE WHEN...) to count total applicants who entered valid nationality information. And I used NULLIF(...,0) to avoid the denominator is 0. The second line makes sure that if no data was found.

Question3:

- What are the average scores for GPA, GRE Quantitative, GRE verbal and GRE

Analytical Writing across all applicants?

- Final result:
 - Average GPA: 3.49
 - Average GRE Quantitative: 163.00
 - Average GRE verbal: 158.00
 - Average GRE Analytical Writing: 4.00
- SQL Query:

```
# Q3
cur.execute("SELECT AVG(gpa), AVG(gre), AVG(gre_v), AVG(gre_aw) FROM applicants;")
row3 = cur.fetchone()
avg_gpa = row3[0] if row3 and row3[0] is not None else 0.0
avg_gre = row3[1] if row3 and row3[1] is not None else 0.0
avg_gre_v = row3[2] if row3 and row3[2] is not None else 0.0
avg_gre_aw = row3[3] if row3 and row3[3] is not None else 0.0

print(f"Average GPA: {avg_gpa:.2f}")
print(f"Average GRE Quantitative: {avg_gre:.2f}")
print(f"Average GRE Verbal: {avg_gre_v:.2f}")
print(f"Average GRE Analytical Writing: {avg_gre_aw:.2f}")
```

- Short explanation:

I used (SELECT AVG(...)) to calculate the mean scores for GPA and all three GRE sections. And to handle cases where no matching data is found, I used following data that can assign a default fallback value of 0.0 if the database returns none, so that we can prevent potential error.

Question4:

- What is the average GPA of American applicants for Fall 2026?
- Final result: 3.66
- SQL Query:

```
# Q4
cur.execute("""
    SELECT AVG(gpa)
    FROM applicants
    WHERE term ILIKE '%fall 2026%'
    AND us_or_international ILIKE '%american%';
""")
q4_gpa = cur.fetchone()[0] or 0.0
print(f"Q4 - Average GPA of American applicants (Fall 2026): {q4_gpa:.2f}")
```

- Short explanation:

I used SELECT AVG... WHERE to calculate the average score of GPA for 2026 Fall American applicants, and I used ILIKE to ignore case. Besides, I use Fetchone to make sure that if no matching records are found, the result can default to 0.0.

Question5:

- What is the acceptance percentage for Fall 2025 applicants?
- Final result: 0.00%
- SQL Query:

```
# Q5
cur.execute("""
    SELECT AVG(CASE WHEN status ILIKE '%accepted%' THEN 100.0 ELSE 0.0 END)
    FROM applicants
    WHERE term ILIKE '%fall 2025%';
""")
q5_pct = cur.fetchone()[0] or 0.0
print(f"Q5 - Fall 2025 acceptance percentage: {q5_pct:.2f}%")
```

- Short explanation:
I assigned 100 for accepted applicants and 0 otherwise, and I calculated the average so that I can directly obtain the percentage number. Although the source website contains data for Fall 2025, the database query returns 0.00 because these records are missing in the database. It is uncertain whether the 30000-record scraping threshold was exhausted before reaching Fall 2025 entries or if there was some mistakes occurred during module 2 assignment. Nevertheless, the SQL query handles the missing entries and default to 0.0.

Question6:

- What is the average GPA of accepted applicants for Fall 2026?
- Final result: 3.19
- SQL Query:

```
# Q6
cur.execute("""
    SELECT AVG(gpa)
    FROM applicants
    WHERE term ILIKE '%fall 2026%'
    AND status ILIKE '%accepted%';
""")
q6_gpa = cur.fetchone()[0] or 0.0
print(f"Q6 - Average GPA of accepted applicants (Fall 2026): {q6_gpa:.2f}")
```

- Short explanation:
Since we need data from accepted applicants for Fall 2026, thus, I used WHERE AND function to filter out the data that meet these 2 conditions. And I used ILIKE to ignore case.

Question7:

- How many total applicants applied for the master's program in Computer Science at Johns Hopkins University (JHU)?

- Final result: 0.00
- SQL Query:

```
# Q7
cur.execute("""
SELECT COUNT(*)
FROM applicants
WHERE (program ILIKE '%johns hopkins%' OR program ILIKE '%jhu%')
AND program ILIKE '%computer science%'
AND (degree ILIKE '%master%' OR degree ILIKE '%ms%' OR degree ILIKE '%m.s.%');
""")
q7_count = cur.fetchone()[0] or 0
print(f"Q7 - JHU CS Master's applicants: {q7_count}")
```

- Short explanation:
Since we need to know the number of something, so I used COUNT. And since we have certain conditions that need to be met, thus I used WHERE AND, to filter out data which meet the condition of university (either Johns Hopkins university or jhu) without case sensitivity, condition of program (either computer science or CS) without case sensitivity and condition of degree (either masters or ms or m.s.) without case sensitivity. The query returns 0 maybe it is because of some mistakes occurred in module2 assignment or the 30000 scraped records simply don't contain any applicants for JHU's CS master's program.

Question8:

- Using the original downloaded fields, how many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at Georgetown, MIT, Stanford, or CMU?
- Final result:0
- SQL Query:

```
# Q8:
cur.execute("""
SELECT COUNT(*)
FROM applicants
WHERE term ILIKE '%fall%' AND term ILIKE '%26%'
AND (status ILIKE '%accepted%')
AND (degree ILIKE '%phd%')
AND (program ILIKE '%computer science%' OR program ILIKE '%cs%')
AND (
    university ILIKE '%georgetown university%'
OR university ILIKE '%massachusetts institute of technology%' OR university ILIKE '%mit%'
OR university ILIKE '%stanford university%'
OR university ILIKE '%carnegie mellon university%' OR university ILIKE '%cmu%'
);
""")
q8_count = cur.fetchone()[0] or 0
```

- Short explanation:
I used WHERE AND to filter out data which meet the required condition and use OR to find data that met one of the conditions of university. The query returns to 0 maybe

because of some mistake in module2 assignment or none of the Fall26 accepted CS PhD records belong to these four target university within 30,000 row sample dataset.

Question9:

- Repeat Question 8, but identify the university and program using llm_generated_university and llm_generated_program instead of the original downloaded fields. Report the Question 8 count, the Question 9 count, and the difference between them.
- Final result:0
- SQL Query:

```
Q9:
cur.execute("""
SELECT COUNT(*)
FROM applicants
WHERE term ILIKE '%fall%' AND term ILIKE '%26%'
AND (status ILIKE '%accept%')
AND (degree ILIKE '%phd%')
AND (llm_generated_program ILIKE '%computer science%' OR llm_generated_program ILIKE '%cs%' OR llm_generated_program ILIKE '%computer%'
AND (
    llm_generated_university ILIKE '%georgetown university%'
    OR llm_generated_university ILIKE '%massachusetts institute of technology%' OR llm_generated_university ILIKE '%mit%'
    OR llm_generated_university ILIKE '%stanford university%'
    OR llm_generated_university ILIKE '%carnegie mellon university%' OR llm_generated_university ILIKE '%cmu%'
);
""")
q9_count = cur.fetchone()[0] or 0
diff = q9_count - q8_count

print(f"Q8 - Original-field count: {q8_count}")
print(f"Q9 - LLM-field count: {q9_count}")
print(f"Q9 - Difference: {diff:+d}")
```

- Short explanation:
I repeated the logic from question8 but switch the university and program fields to the LLM columns. However, since many LLM-generated fields defaulted to 'Unknown' which may cause by some mistakes occurred in module2 assignment, so LLM extraction could not recover non-existent data.

Question10:

- What is the average GPA for each degree level (e.g., Masters, PhD)?
- Final results:
 - Masters: 3.52
 - PhD: 3.44
- SQL Query:

```
# Q10
print("Q10 - Average GPA by Degree Level:")
cur.execute("""
    SELECT degree, AVG(gpa)
    FROM applicants
    WHERE degree IS NOT NULL AND gpa IS NOT NULL
    GROUP BY degree;
""")
for row in cur.fetchall():
    deg = row[0]
    gpa_val = row[1] or 0.0
    print(f" {deg}: {gpa_val:.2f}")
```

- Short explanation: I used AVG to calculate the average GPA and I used GROUP function to divide applicants into master and phd groups so that we can calculate the average gpa separately.

Question11:

- What is the acceptance rate for international applicants for the Spring 2027 term?
- Final results: 37.5%
- SQL Query:

```
# Q11
cur.execute("""
    SELECT
        100.0 * COUNT(CASE WHEN status ILIKE '%accepted%' THEN 1 END) /
        NULLIF(COUNT(*), 0)
    FROM applicants
    WHERE us_or_international ILIKE '%international%'
    AND term ILIKE '%spring 2027%';
""")
q11_pct = cur.fetchone()[0] or 0.0
print(f"Q11 - Spring 2027 International Acceptance Rate: {q11_pct:.2f}%")
```

- Short explanation:
I used COUNT to calculate the total number of accepted applicants and multiplies by 100 so that we can obtain percentage number, and I divided it by total matching international applicants, I used NULLIF to convert a 0 count to NULL to avoid errors. I used WHERE function to filter out international applicants only and added the condition of term of spring 2027.